

Exam Summary

Module Name

Student

Last updated: 23. April 2026

Table of Contents

1. Week 1: Linear Algebra for Deep Learning	3
1.1. Chapter 1: Linear Algebra for Deep Learning	3
2. Week 2: Probability and Information Theory	13
2.1. Chapter 2: Probability and Information Theory	13
3. Week 3	18
3.1. Woche 3: Numerische Berechnung & Gradientenbasierte Optimierung	18
4. Überblick	19
4.1. 1. Numerische Probleme	19
4.2. 2. Gradientenbasierte Optimierung	20
4.3. 3. Jacobi-Matrix	22
4.4. 4. Hessian-Matrix	22
4.5. 5. Newton's Method (Zweiter-Ordnung-Optimierung)	24
4.6. 6. Taylor-Approximation zweiter Ordnung	25
4.7. 7. Konditionszahl der Hessian und Optimierung	26
4.8. 8. Vollständiges Lösungsrezept für Optimierungsaufgaben	26
5. Week 4	27
5.1. Week 4: Machine Learning Basics — Generalisierung, Bias, Varianz	27
6. Lineare Regression als Einstieg	28
7. I.I.D.-Annahme und warum $\text{MSE}_{\text{train}}$ sinnvoll ist	29
8. Kapazität, Overfitting und Underfitting	30
9. Regularisierung	31
10. Hyperparameter und Datensatz-Splits	32
11. Schätzer, Bias und Varianz	33
11.1. Bias eines Schätzers	33
11.2. Varianz und Standardfehler	33
11.3. Beispiel: Gausssche Varianz-Schätzer	34
11.4. Konsistenz	34
12. Bias-Varianz-Tradeoff	35
13. Aufgaben-Lösungsrezepte für die Prüfung	36
14. No Free Lunch Theorem	38

Week 1: Linear Algebra for Deep Learning

Chapter 1: Linear Algebra for Deep Learning

Part 1 — The Building Blocks

Type	Dims	Notation	Example
Scalar	0D	a	single loss value
Vector	1D	$\mathbf{x}$ — always column	$[1, 2, 3]^\top$
Matrix	2D	$\mathbf{A}$, element $A_{i,j}$	weight layer
Tensor	3D+	$\mathbf{A}$, element $A_{i,j,k}$	image batch

Column vector and matrix notation:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad \mathbf{A} = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{pmatrix}$$

Part 2 — Operations

Matrix addition (same size only):

- Used for: combining two sets of features, adding residuals in ResNets

$$C_{i,j} = A_{i,j} + B_{i,j}$$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

Scalar on matrix (a = scale, c = bias/shift):

- Used for: normalising data, applying a weight + bias to a whole layer

$$D_{i,j} = a \cdot B_{i,j} + c$$

$$2 \cdot \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + 10 = \begin{pmatrix} 12 & 14 \\ 16 & 18 \end{pmatrix} \rightarrow \text{every element } \times 2, \text{ then } +10$$

Broadcasting (vector added to every column of matrix):

- Used for: adding a bias vector to an entire batch of inputs at once

$$C_{i,j} = A_{i,j} + b_j$$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 \\ 20 \end{pmatrix} = \begin{pmatrix} 11 & 22 \\ 13 & 24 \\ 15 & 26 \end{pmatrix}$$

Matrix multiplication:

- *Used for:* the core forward pass — transforming inputs through a weight layer: $\mathbf{y} = W\mathbf{x}$

$$\mathbf{C} = \mathbf{AB} \quad C_{i,j} = \sum_k A_{i,k} \cdot B_{k,j}$$

Shape: $(m \times k) \cdot (k \times n) = (m \times n)$ — inner dims must match.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

Inner / dot product (vectors only, result is a scalar):

- *Used for:* measuring similarity between two vectors; attention scores in Transformers

$$\mathbf{x}^\top \mathbf{y} = \sum_i x_i y_i = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos \theta$$

$$(1 \ 2 \ 3) \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$$

Hadamard (element-wise) product:

- *Used for:* gating mechanisms (LSTM, GRU), applying masks, dropout

$$\mathbf{C} = \mathbf{A} \odot \mathbf{B} \quad C_{i,j} = A_{i,j} \cdot B_{i,j}$$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \odot \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 5 & 12 \\ 21 & 32 \end{pmatrix}$$

Properties of matrix multiplication:

- Distributive: $A(B + C) = AB + AC$
- Associative: $A(BC) = (AB)C$

Exam Trap: $AB \neq BA$ — matrix multiplication is **NOT commutative**. This is on every exam.

Transpose:

- *Used for:* converting column↔row vectors, computing dot products ($\mathbf{x}^\top \mathbf{y}$), flipping weight matrices in backpropagation

$$(A^\top)_{i,j} = A_{j,i} \quad (AB)^\top = B^\top A^\top$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}^\top = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix} \rightarrow (2 \text{ times } 3) \text{ becomes } (3 \text{ times } 2)$$

Part 3 — Norms

- *Used for:* measuring magnitude of weights, gradients, errors; regularisation (L1/L2); comparing vectors

General L^p norm:

$$\|x\|_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}, \quad p \geq 1$$

Example with $x = [3, -4]^\top$:

$$\|x\|_1 = 3 + 4 = 7 \quad \|x\|_2 = \sqrt{9 + 16} = 5 \quad \|x\|_\infty = 4$$

Norm	Formula	Use
L^2 (Euclidean)	$\ x\ _2 = \sqrt{\sum_i x_i^2}$	most common
Squared L^2	$\ x\ _2^2 = x^\top x$	easier to compute
L^1	$\ x\ _1 = \sum_i x_i $	sparsity
L^0	count of non-zeros	not a true norm
L^∞	$\ x\ _\infty = \max_i x_i $	max element
Frobenius	$\ A\ _F = \sqrt{\sum_{i,j} A_{i,j}^2} = \sqrt{\text{tr}(AA^\top)}$	matrix size

Quick Reference — A vs $A^\top$ vs A^{-1}

Notation	Name	What it does	How it looks
A	Original matrix	Transforms input	unchanged
$A^\top$	Transpose	Swaps rows and columns	$(A^\top)_{i,j} = A_{j,i}$
A^{-1}	Inverse	Undoes A — like dividing	$A^{-1}A = I$

Starting with the same matrix A :

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Transpose $A^\top$ — flip across diagonal (rows become columns):

$$A^\top = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \rightarrow \text{row 1 } [1, 2] \text{ becomes col 1 } \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Inverse A^{-1} — the “undo” matrix, found by calculation (not just flipping!):

$$A^{-1} = \begin{pmatrix} -2 & 1 \\ 3/2 & -1/2 \end{pmatrix} \rightarrow A^{-1}A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

Special case — Orthogonal matrix: transpose = inverse ($A^\top = A^{-1}$)
 $\rightarrow$ only works because orthogonal matrices purely rotate (no stretching).

Exam Trap: $A^\top$ is always easy — just flip.

A^{-1} requires real computation — unless A is orthogonal (then $A^{-1} = A^\top$ for free).

Part 4 — Identity, Inverse, Solving Systems

- Used for: solving systems of equations $Ax = b$; finding model parameters given data

$$A^{-1}A = I \quad Ax = b \Rightarrow x = A^{-1}b$$

Example — solve $Ax = b$:

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix} \quad b = \begin{pmatrix} 6 \\ 8 \end{pmatrix} \rightarrow A^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/4 \end{pmatrix} \rightarrow x = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

How to compute A^{-1} — 2×2 formula (exam):

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \Rightarrow A^{-1} = \frac{1}{\det(A)} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \quad \text{where } \det(A) = ad - bc$$

Steps: ① swap a and d on diagonal ② flip signs of b and c ③ divide by $\det(A)$

Example:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \rightarrow \det(A) = 1 \cdot 4 - 2 \cdot 3 = -2 \rightarrow A^{-1} = \frac{1}{-2} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ 3/2 & -1/2 \end{pmatrix}$$

$$\text{Verify: } A^{-1}A = \begin{pmatrix} -2 & 1 \\ 3/2 & -1/2 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \checkmark$$

Exam Trap: If $\det(A) = ad - bc = 0 \rightarrow$ division by zero $\rightarrow$ **no inverse exists** (singular matrix).

For larger matrices: use Gauss-Jordan — write $[A \mid I]$, row-reduce until left side $= I$, right side becomes A^{-1} .

A^{-1} exists only if: A is square AND columns are linearly independent (non-singular).

Situation	Solutions
$m > n$ (overdetermined)	None (generically)
$m < n$ (underdetermined)	Infinitely many
$m = n$, independent cols	Exactly one

Part 5 — Special Matrices

Diagonal:

- Used for: efficient scaling; eigenvalue matrices in decompositions; fast inversion

$$\text{diag}(v)x = v \odot x \quad \text{diag}(v)^{-1} = \text{diag}([1/v_1, \dots, 1/v_n]^T)$$

$$\text{diag}\left(\begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix}\right)x = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 2x_1 \\ 3x_2 \\ 4x_3 \end{pmatrix} \rightarrow \text{same as } \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} \odot x$$

Symmetric:

- Used for: covariance matrices, distance matrices, any matrix of pairwise values

$$A = A^T$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 5 & 4 \\ 3 & 4 & 6 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 5 & 4 \\ 3 & 4 & 6 \end{pmatrix}^T \rightarrow \text{mirrored across diagonal}$$

Orthogonal (rows AND cols are orthonormal):

- *Used for:* rotation matrices, V and U in SVD, efficient inversion (just transpose)

Exam Trap: Symmetric $\neq$ Orthogonal — completely different properties!

Symmetric: $A = A^T$ (mirrored across diagonal).

Orthogonal: $A^T A = I$ (columns perpendicular + unit length). A matrix can be orthogonal without looking symmetric at all.

How to check if a matrix is orthogonal — verify two conditions on every column:

1. **Unit length:** $\|c_i\|_2 = 1$ — each column has length 1
2. **Perpendicular:** $c_i^T c_j = 0$ for all $i \neq j$ — every pair of columns is at 90°

If both hold $\rightarrow$ orthogonal $\rightarrow$ use $A^{-1} = A^T$ (free inverse, no computation).

Example — check col 1 and col 2 of $A = \frac{1}{3} \begin{pmatrix} 2 & -2 & 1 \\ 1 & 2 & 2 \\ 2 & 1 & -2 \end{pmatrix}$:

First, extract the columns (each column = one vertical slice of the matrix):

$$c_1 = \frac{1}{3} \begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix} \quad c_2 = \frac{1}{3} \begin{pmatrix} -2 \\ 2 \\ 1 \end{pmatrix} \quad c_3 = \frac{1}{3} \begin{pmatrix} 1 \\ 2 \\ -2 \end{pmatrix}$$

Check unit length — square each element, sum them, take the square root. Must equal 1:

$$\|c_1\|_2 = \sqrt{\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2} = \sqrt{\frac{4}{9} + \frac{1}{9} + \frac{4}{9}} = \sqrt{\frac{9}{9}} = \sqrt{1} = 1 \checkmark$$

Check perpendicular — dot product = multiply matching elements, then sum. Must equal 0:

$$\begin{aligned} c_1^T c_2 &= \frac{2}{3} \cdot \frac{-2}{3} + \frac{1}{3} \cdot \frac{2}{3} + \frac{2}{3} \cdot \frac{1}{3} \\ &= \frac{-4}{9} + \frac{2}{9} + \frac{2}{9} = \frac{-4 + 2 + 2}{9} = \frac{0}{9} = 0 \checkmark \end{aligned}$$

Same check for every other pair: $c_1^T c_3 = 0$ and $c_2^T c_3 = 0$ (verify yourself!).

$$A^T A = A A^T = I \Rightarrow A^{-1} = A^T$$

Inverting an orthogonal matrix = just transposing it — no computation needed.

Part 6 — Eigendecomposition

- *Used for:* understanding what a matrix does geometrically; PCA; finding principal directions of data; quadratic optimisation

$$A v = \lambda v$$

- v = eigenvector (direction unchanged by A , unit length $\|v\| = 1$)

- λ = eigenvalue (scaling factor along that direction)

How to compute eigenvalues — solve the characteristic equation $\det(A - \lambda I) = 0$:

Step 1: Subtract λ from every diagonal element $\rightarrow$ get $A - \lambda I$

Step 2: Compute $\det(A - \lambda I) = 0 \rightarrow$ gives a polynomial in λ

Step 3: Solve for λ using the quadratic formula (for 2×2):

$$\lambda = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Example — find eigenvalues of $B = \begin{pmatrix} 3 & 12 \\ 12 & 56 \end{pmatrix}$:

$$\text{Step 1: } B - \lambda I = \begin{pmatrix} 3 - \lambda & 12 \\ 12 & 56 - \lambda \end{pmatrix}$$

$$\text{Step 2: } \det = (3 - \lambda)(56 - \lambda) - 12 \cdot 12 = \lambda^2 - 59\lambda + 24 = 0$$

$$\text{Step 3: } \lambda = \frac{59 \pm \sqrt{59^2 - 4 \cdot 24}}{2} = \frac{59 \pm \sqrt{3385}}{2} \rightarrow \lambda_1 \approx 58.59, \quad \lambda_2 \approx 0.41$$

Example — $A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ has eigenvalues $\lambda_1 = 4, \lambda_2 = 2$:

$$A \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix} = 4 \begin{pmatrix} 1 \\ 1 \end{pmatrix} \rightarrow \text{direction } [1, 1]^\top \text{ scaled by } 4$$

$$A \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 \\ -2 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ -1 \end{pmatrix} \rightarrow \text{direction } [1, -1]^\top \text{ scaled by } 2$$

Full decomposition:

$$A = V \text{diag}(\lambda) V^{-1} \quad V = [\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}]$$

Intuition: **rotate** (by V) $\rightarrow$ **scale** (by λ) $\rightarrow$ **rotate back** (V^{-1}).

For real symmetric matrices: n real eigenvalues, orthogonal eigenvectors $\rightarrow V^{-1} = V^\top$.

Optimization (exam!):

$$\max_{\|\mathbf{x}\|=1} \mathbf{x}^\top A \mathbf{x} \rightarrow \mathbf{x} = \text{eigenvector of max eigenvalue}$$

$$\min_{\|\mathbf{x}\|=1} \mathbf{x}^\top A \mathbf{x} \rightarrow \mathbf{x} = \text{eigenvector of min eigenvalue}$$

Part 7 — Singular Value Decomposition (SVD)

- *Used for:* pseudoinverse, data compression, dimensionality reduction (PCA), recommender systems, solving any linear system

Works for **any** matrix (generalization of eigendecomposition):

$$A = U D V^\top$$

- U : orthogonal — left singular vectors (output directions)

- D : diagonal — singular values (how much each direction is stretched)
- V : orthogonal — right singular vectors (input directions)

Full worked example — SVD of $A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix}$ (shape 3×2):

Step 1 — Compute $A^\top A$, find eigenvalues (used for both D and V):

$$A^\top A = \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 9 & 0 \\ 0 & 4 \\ 0 & 0 \end{pmatrix} \rightarrow \lambda_1 = 9, \quad \lambda_2 = 4$$

Step 2 — Build D (square roots, same shape as A):

$$D = \begin{pmatrix} \sqrt{9} & 0 \\ 0 & \sqrt{4} \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix}$$

Step 3 — Build V (solve $(A^\top A - \lambda I)\mathbf{v} = 0$ for each λ , stack as columns):

$$\lambda_1 = 9: \begin{pmatrix} 0 & 0 \\ 0 & -5 \end{pmatrix} \mathbf{v} = 0 \rightarrow \mathbf{v}^{(1)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \lambda_2 = 4: \begin{pmatrix} 5 & 0 \\ 0 & 0 \end{pmatrix} \mathbf{v} = 0 \rightarrow \mathbf{v}^{(2)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$V = [\mathbf{v}^{(1)} \mid \mathbf{v}^{(2)}] = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Step 4 — Compute $AA^\top$, find eigenvectors (for U):

$$AA^\top = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 9 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 0 \end{pmatrix} \rightarrow \lambda_1 = 9, \lambda_2 = 4, \lambda_3 = 0$$

$$\mathbf{u}^{(1)} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \mathbf{u}^{(2)} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \mathbf{u}^{(3)} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$U = [\mathbf{u}^{(1)} \mid \mathbf{u}^{(2)} \mid \mathbf{u}^{(3)}] = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Verify: $UDV^\top = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} = A \checkmark$

Tip: Key insight: eigenvectors = directions. Stack them side by side as columns — col 1 = biggest λ , col 2 = next, and so on.

How to build U — eigenvectors of $AA^\top$, stacked as columns in descending λ order. Size: $m \times m$ (m = rows of A):

$$U = [\mathbf{u}^{(1)} \mid \mathbf{u}^{(2)} \mid \dots \mid \mathbf{u}^{(m)}] \quad \text{col } i = \text{eigenvec of } AA^\top \text{ for } \lambda_i$$

Example — A is $3 \times 2 \rightarrow AA^\top$ is $3 \times 3 \rightarrow U$ is 3×3 :

$$U = \begin{pmatrix} -0.283 & -0.868 & 0.408 \\ -0.538 & -0.208 & -0.816 \\ -0.794 & 0.451 & 0.408 \end{pmatrix}$$

How to build D — $\sqrt{\lambda_i}$ of $A^\top A$ on diagonal, descending order. Size = same shape as A :

$$D = \begin{pmatrix} \sqrt{\lambda_1} & 0 & \dots \\ 0 & \sqrt{\lambda_2} & \dots \\ \vdots & \vdots & \ddots \end{pmatrix} \text{ same shape as } A, \text{ extra rows/cols} = 0$$

Example — A is 3×2 , $\lambda_1 = 58.59$, $\lambda_2 = 0.41$:

$$D = \begin{pmatrix} \sqrt{58.59} & 0 \\ 0 & \sqrt{0.41} \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 7.65 & 0 \\ 0 & 0.64 \\ 0 & 0 \end{pmatrix}$$

How to build V — eigenvectors of $A^\top A$, stacked as columns in descending λ order. Size: $n \times n$ ($n = \text{cols of } A$):

$$V = [\mathbf{v}^{(1)} \mid \mathbf{v}^{(2)} \mid \dots \mid \mathbf{v}^{(n)}] \quad \text{col } i = \text{eigenvector of } A^\top A \text{ for } \lambda_i$$

Example — A has 2 cols $\rightarrow V$ is 2×2 :

$$V = \begin{pmatrix} -0.211 & -0.977 \\ -0.977 & 0.211 \end{pmatrix}$$

Tip: Full SVD recipe for $A = UDV^\top$:

1. U : eigenvectors of $AA^\top$ as columns ($m \times m$)
2. D : $\sqrt{\lambda_i}$ on diagonal, same shape as A ($m \times n$)
3. V : eigenvectors of $A^\top A$ as columns ($n \times n$)

All in descending eigenvalue order.

Part 8 — Moore-Penrose Pseudoinverse

- *Used for:* least-squares regression, solving overdetermined systems (more equations than unknowns); any time a true inverse doesn't exist

When A^{-1} doesn't exist:

$$A^+ = VD^+U^\top \quad (D^+ : \text{reciprocal of each non-zero diagonal of } D)$$

Example — $D = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} \rightarrow D^+ = \begin{pmatrix} 1/3 & 0 & 0 \\ 0 & 1/2 & 0 \end{pmatrix}$

Case	Result
More rows than cols (overdetermined)	Minimises $\ A\mathbf{x} - \mathbf{b}\ _2$ (least squares)
More cols than rows (underdetermined)	Minimises $\ \mathbf{x}\ _2$ among all solutions

Part 9 — Trace

- *Used for:* computing Frobenius norm, simplifying matrix derivative expressions, checking eigenvalue sums

$$\text{tr}(A) = \sum_i A_{i,i} = \sum_i \lambda_i$$

Example:

$$\text{tr}\left(\begin{pmatrix} 1 & 9 & 7 \\ 2 & 5 & 3 \\ 4 & 6 & 8 \end{pmatrix}\right) = 1 + 5 + 8 = 14$$

- $\text{tr}(A^\top) = \text{tr}(A)$
- $\text{tr}(ABC) = \text{tr}(CAB) = \text{tr}(BCA) \leftarrow$ cyclic invariant
- $\|A\|_F = \sqrt{\text{tr}(AA^\top)}$

Part 10 — Determinant

- *Used for:* checking if a matrix is invertible; measuring volume scaling; appears in probability density formulas (multivariate Gaussian)

$$\det(A) = \prod_i \lambda_i$$

Example:

$$\det\left(\begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}\right) = 3 \cdot 3 - 1 \cdot 1 = 8 \rightarrow \lambda_1 \cdot \lambda_2 = 4 \cdot 2 = 8 \checkmark$$

- $\det(A) = 0 \rightarrow$ singular, not invertible (at least one eigenvalue is zero)
- $\det(A) \neq 0 \rightarrow$ invertible
- Geometric meaning: factor by which A scales volume

Cheat Sheet**WEEK 1 — LINEAR ALGEBRA CHEAT SHEET**

Structures: scalar=0D · vector=1D(col) · matrix=2D · tensor=3D+

Multiply:

- Standard: $C_{i,j} = \sum_k A_{i,k} B_{k,j}$
- Dot product: $\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$
- Hadamard $\odot$: $C_{i,j} = A_{i,j} B_{i,j}$
- $AB \neq BA$ — NEVER commutative

Norms: $L^2 = \sqrt{\sum x^2}$ · $L^1 = \sum |x|$ · $L^\infty = \max |x|$ · $\|A\|_F = \sqrt{\text{tr}(AA^\top)}$

Special: Symmetric $A = A^\top$ · Orthogonal $A^{-1} = A^\top$ · Diagonal: invert = reciprocal diagonal

Eigen: $A\mathbf{v} = \lambda\mathbf{v} \rightarrow A = V \text{diag}(\lambda) V^{-1}$

SVD: $A = U D V^\top$ (always works)

Pseudoinverse: $A^+ = V D^+ U^\top$

Trace: $\text{tr}(A) = \sum A_{i,i} = \sum \lambda_i$ (cyclic invariant)

Det: $\det(A) = \prod \lambda_i$ · zero = singular = not invertible

Week 2: Probability and Information Theory

Chapter 2: Probability and Information Theory

Part 1 — Random Variables

- *Used for:* describing uncertain quantities; modelling data distributions; defining loss functions

A **random variable** x can take on different values with associated probabilities.

Type	Values	Described by	Notation
Discrete	Countable set	Probability Mass Function (PMF)	$P(x = k)$
Continuous	Real interval	Probability Density Function (PDF)	$p(x)$

PMF (discrete) — probability at each exact value:

$$P(x = k) \geq 0 \quad \sum_k P(x = k) = 1$$

PDF (continuous) — probability density; integrate over an interval to get probability:

$$p(x) \geq 0 \quad \int_{-\infty}^{\infty} p(x) dx = 1 \quad P(a \leq x \leq b) = \int_a^b p(x) dx$$

Exam Trap: $p(x)$ is a **density**, not a probability. $p(x) > 1$ is valid! Only the integral over an interval gives probability.

Part 2 — Key Distributions

Bernoulli — single binary outcome ($x \in \{0, 1\}$):

- *Used for:* modelling coin flips, binary classification outputs, gates in LSTMs

$$P(x = 1) = \varphi, \quad P(x = 0) = 1 - \varphi$$

$$\mathbb{E}[x] = \varphi \quad \text{Var}(x) = \varphi(1 - \varphi)$$

Example: $\varphi = \frac{3}{4} \rightarrow \mathbb{E}[x] = \frac{3}{4}, \text{Var}(x) = \frac{3}{16}$.

Uniform (continuous) — equal probability over interval $[a, b]$:

- *Used for:* initialising weights, random sampling, baseline distributions

$$p(x) = \frac{1}{b - a} \quad x \in [a, b], \quad \text{else } 0$$

$$\mathbb{E}[x] = \frac{a + b}{2} \quad \text{Var}(x) = \frac{(b - a)^2}{12}$$

Derived from integration: $\mathbb{E}[x] = \int_a^b x \cdot \frac{1}{b-a} dx = \frac{a+b}{2}$ (the midpoint of $[a, b]$)

Example: $x \in [-0.5, 1] \rightarrow p(x) = \frac{2}{3}, \mathbb{E}[x] = \frac{1}{4}, \text{Var}(x) = \frac{3}{16}$.

Gaussian (Normal) — bell curve, most important distribution in ML:

- *Used for:* modelling real-valued data, noise, weight initialisation, latent spaces in VAEs

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma^2} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$$\mathbb{E}[x] = \mu \quad \text{Var}(x) = \sigma^2$$

Standard Normal: $\mu = 0, \sigma = 1$. Notation: $x \sim \mathcal{N}(\mu, \sigma^2)$.

Tip: Why Gaussian? The Central Limit Theorem: sums of many independent random variables converge to a Gaussian. Most real-world noise is approximately Gaussian.

Exponential — models time between events; heavy tail to the right:

$$p(x) = \lambda \exp(-\lambda x), \quad x \geq 0 \quad \mathbb{E}[x] = \frac{1}{\lambda}$$

Laplace — like Gaussian but with heavier tails; used in L1 regularisation:

$$p(x) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right)$$

Part 3 — Expectations and Variance

- *Used for:* computing loss function values, quantifying spread, deriving gradient updates

Discrete: $\mathbb{E}[f(x)] = \sum_x f(x)P(x)$

Continuous: $\mathbb{E}[f(x)] = \int f(x)p(x)dx$

Variance: $\text{Var}(x) = \mathbb{E}[x^2] - (\mathbb{E}[x])^2 = \mathbb{E}[(x - \mathbb{E}[x])^2]$

Key properties:

- $\mathbb{E}[ax + b] = a\mathbb{E}[x] + b$ — linearity
- $\text{Var}(ax) = a^2 \text{Var}(x)$
- $\text{Var}(x + y) = \text{Var}(x) + \text{Var}(y)$ only if x, y are independent

Covariance — measures linear dependence between two variables:

$$\text{Cov}(x, y) = \mathbb{E}[(x - \mathbb{E}[x])(y - \mathbb{E}[y])] = \mathbb{E}[xy] - \mathbb{E}[x]\mathbb{E}[y]$$

Part 4 — Bayes' Theorem

- *Used for:* updating beliefs given evidence; Naive Bayes classifiers; probabilistic inference in all Bayesian models

$$P(A | B) = \frac{P(B | A), P(A)}{P(B)}$$

- $P(A)$ — **prior**: belief before seeing evidence
- $P(B | A)$ — **likelihood**: probability of evidence given hypothesis
- $P(A | B)$ — **posterior**: updated belief after evidence
- $P(B)$ — **marginal** (normalisation constant)

Law of total probability (for computing $P(B)$):

$$P(B) = \sum_i P(B | A_i) P(A_i)$$

Example — cab problem: 5000 yellow cabs, 100 red cabs; witness accuracy 95%. Find $P(c = r | w = r)$:

$$P(c = r) = \frac{100}{5100} \approx 0.0196$$

$$P(w = r) = 0.95 \cdot \frac{100}{5100} + 0.05 \cdot \frac{5000}{5100} \approx 0.0676$$

$$P(c = r | w = r) = \frac{0.0196 \cdot 0.95}{0.0676} \approx 27.5\%$$

Exam Trap: Base rate neglect: even with a 95%-accurate witness, the answer is only 27.5%. Red cabs are so rare that most “red” reports are still wrong yellow cabs. Always account for the prior!

Bayesian classification — assign x to the class with highest posterior:

$$\text{assign to } c_i \quad \text{if} \quad P(y = c_i) p(x | y = c_i) > P(y = c_j) p(x | y = c_j)$$

The denominator $p(x)$ is the same for all classes and cancels.

Part 5 — Information Theory

- *Used for:* cross-entropy loss (classifier training), measuring model uncertainty, defining KL divergence for VAEs

Self-information — surprise of a single outcome:

$$I(x) = -\log P(x)$$

Rare events $\rightarrow$ large $I(x)$ (more surprising). Common events $\rightarrow$ small $I(x)$.

Log base: $\log_2$ = bits; $\ln$ = nats. Deep learning typically uses nats.

Shannon Entropy — expected surprise (average information content):

$$\text{Discrete:} \quad H(x) = -\sum_x P(x) \log P(x) = \mathbb{E}[-\log P(x)]$$

$$\text{Gaussian:} \quad H(x) = \frac{1}{2}(\log(2\pi\sigma^2) + 1)$$

Higher variance $\rightarrow$ higher entropy. Uniform distribution has maximum entropy.

Example:

- $\sigma_1^2 = 0.26 \rightarrow H(x \mid y = c_1) = 0.5(\ln(2\pi \cdot 0.26) + 1) \approx 0.745$ nats
- $\sigma_2^2 = 0.99 \rightarrow H(x \mid y = c_2) = 0.5(\ln(2\pi \cdot 0.99) + 1) \approx 1.414$ nats

Cross-Entropy — entropy of distribution P measured under model Q :

$$H(P, Q) = - \sum_x P(x) \log Q(x) = H(P) + D_{\text{KL}}(P \parallel Q)$$

Used as the standard loss for classification (comparing true labels P to model predictions Q).

KL Divergence — measures how much Q differs from P :

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \geq 0$$

Exam Trap: $D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P)$ — KL divergence is **asymmetric**. It is not a distance metric!

- $D_{\text{KL}}(P \parallel Q) = 0$ iff $P = Q$ everywhere.
- Minimising $H(P, Q)$ over Q is equivalent to minimising $D_{\text{KL}}(P \parallel Q)$ (since $H(P)$ is constant).

Tip: Cross-entropy loss in practice:

$$\mathcal{L} = - \sum_k y_k \log \hat{y}_k$$

where y_k = true label (one-hot), $\hat{y}_k$ = softmax output. This is exactly $H(P, Q)$.

Cheat Sheet

WEEK 2 — PROBABILITY & INFORMATION THEORY CHEAT SHEET

Random Variables:

- Discrete $\rightarrow$ PMF: $\sum P(x) = 1$
- Continuous $\rightarrow$ PDF: $\int p(x)dx = 1$; $p(x)$ can exceed 1!

Key Distributions:

- Bernoulli(φ): $\mathbb{E} = \varphi$, $\text{Var} = \varphi(1 - \varphi)$
- Uniform[a, b]: $p = \frac{1}{b-a}$, $\mathbb{E} = \frac{a+b}{2}$, $\text{Var} = \frac{(b-a)^2}{12}$
- Gaussian(μ, σ^2): $p = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$

Expectation & Variance:

- $\mathbb{E}[f(x)] = \sum f(x)P(x)$ or $\int f(x)p(x)dx$
- $\text{Var}(x) = \mathbb{E}[x^2] - (\mathbb{E}[x])^2$

Bayes: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ — always account for the prior!

Classification: assign to c_i if $P(c_i)p(x|c_i)$ is largest.

Entropy:

- Discrete: $H = -\sum P \log P$
- Gaussian: $H = \frac{1}{2}(\log(2\pi\sigma^2) + 1)$

Cross-entropy: $H(P, Q) = -\sum P \log Q = H(P) + D_{\text{KL}}(P\|Q)$

KL-Divergence: $D_{\text{KL}}(P\|Q) = \sum P \log \frac{P}{Q} \geq 0$; asymmetric!

Week 3

Woche 3: Numerische Berechnung & Gradientenbasierte Optimierung

Überblick

Diese Woche verbindet zwei Themen: **numerische Stabilität** (was kann beim Rechnen auf dem Computer schiefgehen?) und **gradientenbasierte Optimierung** (wie minimiert man eine Verlustfunktion?). Das Herzstück des Deep Learning — das Trainieren eines Netzes — ist ein Optimierungsproblem, das mit diesen Werkzeugen gelöst wird.

1. Numerische Probleme

1.1 Quantisierung und Rundungsfehler

Kernidee: Computer können keine beliebigen reellen Zahlen speichern — nur endlich viele diskrete Werte. Jede Zahl wird auf den nächsten darstellbaren Wert gerundet.

Bei b Bits und Wertebereich $[-\frac{R}{2}, \frac{R}{2}]$ gilt:

$$Q = \frac{R}{2^b}$$

Schrittweite Q : je kleiner, desto genauer. Akkumulierte Rundungsfehler über viele Operationen können erheblich sein.

1.2 Underflow und Overflow

Kernidee: Zu kleine Zahlen werden zu 0 gerundet (Underflow), zu grosse zu ∞ (Overflow). Beides kann Folgeberechnungen zerstören.

Exam Trap: Underflow: $x \rightarrow 0$, dann bricht $\frac{1}{x}$ oder $\log(x)$ zusammen (Division durch Null / $\log(0) = -\infty$).

Overflow: $x \rightarrow \infty$, dann ist jede weitere Rechenoperation undefiniert.

Typischer Denkfehler: “Underflow ist harmlos, weil die Zahl einfach null wird.” — Falsch! Die **Folgeoperationen** sind das Problem.

1.3 Stabilisierung der Softmax-Funktion

Kernidee: Die Softmax-Funktion für K Klassen:

$$\text{softmax}(x)_i = \frac{\exp(x_i)}{\sum_{j=1}^K \exp(x_j)}$$

ist anfällig für Over- und Underflow, wenn alle $x_i = c$ mit sehr grossem oder sehr kleinem c .

Tip: Erkennungsmerkmal: Aufgabe fragt nach numerischer Stabilität der Softmax — oder man implementiert sie von Hand ohne Framework.

Stabilisierung: Subtrahiere $k = \max_i(x_i)$ von allen Eingaben:

$$\text{softmax}(\mathbf{x})_i = \frac{\exp(x_i - k)}{\sum_j \exp(x_j - k)}$$

Warum funktioniert das?

- Der grösste Exponent wird zu $\exp(0) = 1$ — kein Overflow.
- Alle anderen Exponenten sind negativ — kein Overflow.
- Der Nenner ist mindestens 1 — kein $\frac{0}{0}$ -Problem.
- Das Ergebnis ist mathematisch identisch (kürzen von $\exp(k)$).

Exam Trap: log-Softmax braucht separate Stabilisierung! Auch nach der Softmax-Stabilisierung kann $\log(\text{softmax}(\mathbf{x})_i)$ noch underflowgefährdet sein, weil stabilisierte Softmax-Werte nahe 0 liegen können. Moderne Frameworks (PyTorch, TensorFlow) behandeln das automatisch.

1.4 Poor Conditioning (Schlechte Konditionierung)

Kernidee: Eine schlecht konditionierte Funktion verstärkt kleine Fehler in der Eingabe zu grossen Fehlern im Output.

Konditionszahl einer Matrix $\mathbf{A}$:

$$\kappa(\mathbf{A}) = \frac{\lambda_{\max}}{\lambda_{\min}}$$

wobei $\lambda_{\max}$ und $\lambda_{\min}$ der grösste bzw. kleinste Eigenwert von $\mathbf{A}$ sind.

- $\kappa \approx 1 \rightarrow$ gut konditioniert, Fehler bleiben gleich gross
- $\kappa \gg 1$ (z.B. > 10000) $\rightarrow$ schlecht konditioniert, Fehler werden verstärkt
- $\mathbf{A}$ und $\mathbf{A}^{-1}$ haben (unter vernünftigen Annahmen) dieselbe Konditionszahl

2. Gradientenbasierte Optimierung

2.1 Kernidee

Training = Minimierung einer Verlustfunktion. Wir minimieren die Kreuzentropie zwischen der empirischen Datenverteilung $\hat{p}_{\text{data}}$ und der Modellverteilung $p_{\text{model}(\boldsymbol{\theta})}$:

$$\min_{\boldsymbol{\theta}} H(\hat{p}_{\text{data}}, p_{\text{model}(\boldsymbol{\theta})})$$

Das Werkzeug dafür: Gradientenabstieg.

2.2 Skalarfall: Ableitung und Gradient Descent

Für $f: \mathbb{R} \rightarrow \mathbb{R}$:

- **Ableitung** $f'(x)$ = Steigung der Funktion an Stelle x
- Negative Ableitung $\rightarrow$ Funktion fällt in positive Richtung
- Nullstelle der Ableitung $\rightarrow$ kritischer Punkt

Update-Regel (Gradient Descent, 1D):

$$x^{(i)} = x^{(i-1)} - \varepsilon \cdot f'(x^{(i-1)})$$

2.3 Vektorfall: Gradient

Für $f : \mathbb{R}^n \rightarrow \mathbb{R}$, Vektor-Eingabe $\mathbf{x}$:

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{pmatrix}$$

Update-Regel (Gradient Descent, n-dimensional):

$$\mathbf{x}^{(i)} = \mathbf{x}^{(i-1)} - \varepsilon \cdot \nabla_{\mathbf{x}} f(\mathbf{x}^{(i-1)})$$

Tip: Woran erkenne ich Gradient Descent in einer Aufgabe?

- “Suche ein Minimum mit der Methode des steilsten Abstiegs”
- “Führe k Iterationen durch, Startpunkt $\mathbf{x}^{(0)}$, Lernrate ε ”
- Gegeben: Verlustfunktion $f(\mathbf{x})$, Startpunkt, Lernrate

Rezept: Steepest Descent (Gradientenabstieg)

1. Berechne den Gradienten $\nabla_{\mathbf{x}} f(\mathbf{x})$ analytisch (alle partiellen Ableitungen)
2. Setze Startpunkt $\mathbf{x}^{(0)}$ ein
3. Berechne $\nabla_{\mathbf{x}} f(\mathbf{x}^{(0)})$
4. Update: $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} - \varepsilon \cdot \nabla_{\mathbf{x}} f(\mathbf{x}^{(0)})$
5. Wiederhole Schritte 3–4 für k Iterationen

Beispiel: $f(\mathbf{x}) = x_1^2 + x_2^2$, $\mathbf{x}^{(0)} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$, $\varepsilon = 0.1$

Gradient: $\nabla f = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix}$

Am Startpunkt: $\nabla f(\mathbf{x}^{(0)}) = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$

Update: $\mathbf{x}^{(1)} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} - 0.1 \cdot \begin{pmatrix} 4 \\ 2 \end{pmatrix} = \begin{pmatrix} 1.6 \\ 0.8 \end{pmatrix}$

2.4 Lernrate ε

- ε zu klein $\rightarrow$ Konvergenz sehr langsam
- ε zu gross $\rightarrow$ Überspringen des Minimums, Divergenz möglich
- Optimale Lernrate: problem- und datenabhängig, wird später automatisch gelernt

2.5 Richtungsableitung

Kernidee: Statt partiell entlang Achsen abzuleiten — Ableitung in **beliebiger Richtung $\mathbf{u}$** (Einheitsvektor):

$$\frac{\partial f(\mathbf{x} + \alpha \mathbf{u})}{\partial \alpha} = \mathbf{u}^T \nabla_{\mathbf{x}} f(\mathbf{x})$$

In welche Richtung soll man gehen, um f am stärksten zu verkleinern?

Mit $\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$ und $\|\mathbf{u}\| = 1$:

$$\min_{\mathbf{u}} \mathbf{u}^T \nabla f = \min_{\mathbf{u}} \|\nabla f\| \cos(\theta)$$

Minimum bei $\cos(\theta) = -1$, also $\theta = 180^\circ \rightarrow$ **entgegen dem Gradienten**.

Tip:

- **Gradient** = Richtung des steilsten Anstiegs
- **Negativer Gradient** = Richtung des steilsten Abstiegs
- Gradient Descent folgt immer dem negativen Gradienten

2.6 Kritische Punkte

Kritische Punkte (stationäre Punkte): $\nabla_{\mathbf{x}} f(\mathbf{x}) = \mathbf{0}$

Gradient Descent terminiert an kritischen Punkten, kann aber in lokalen Minima oder Sattelpunkten stecken bleiben.

3. Jacobi-Matrix

Kernidee: Verallgemeinerung des Gradienten auf vektorwertige Funktionen $f: \mathbb{R}^m \rightarrow \mathbb{R}^n$.

$$\mathbf{J} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_m} \end{pmatrix}$$

Dimension: $n \times m$ (Zeile i = Gradient von $f_{i(\mathbf{x})}$).

Tip: Wann brauche ich die Jacobi-Matrix?

- Funktion hat Vektor-Eingabe **und** Vektor-Ausgabe
- Typische Anwendung: Ableitung der Softmax-Funktion, Backpropagation

4. Hessian-Matrix

4.1 Definition

Kernidee: Die Hessian-Matrix enthält alle zweiten partiellen Ableitungen von $f: \mathbb{R}^n \rightarrow \mathbb{R}$ — sie ist die Jacobi-Matrix des Gradienten.

$$\mathbf{H}(f)(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$

Die Hessian-Matrix ist **symmetrisch** $\rightarrow$ reelle Eigenwerte, Eigenzerlegung möglich:

$$\mathbf{H} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$$

wobei $\mathbf{V}$: Matrix der Eigenvektoren, $\mathbf{\Lambda}$: Diagonalmatrix der Eigenwerte λ_i .

4.2 Krümmung und Eigenwerte

Zweite Richtungsableitung in Richtung d :

$$d^T \mathbf{H} d$$

- Falls d ein Eigenvektor: $d^T \mathbf{H} d = \lambda$ (Eigenwert = Krümmung in dieser Richtung)
- Für beliebige Richtung: $d^T \mathbf{H} d = \sum_i \cos^2(\theta_i) \lambda_i$

Maximale Krümmung = $\lambda_{\max}$ in Richtung $\mathbf{v}_{\max}$

Minimale Krümmung = $\lambda_{\min}$ in Richtung $\mathbf{v}_{\min}$

4.3 Zweiter Ableitungstest

Rezept: Kritischen Punkt klassifizieren

- Berechne Gradient $\nabla f(\mathbf{x})$ und setze $= \mathbf{0}$
- Löse das Gleichungssystem $\rightarrow$ kritische Punkte $\mathbf{c}_i$
- Berechne Hessian $\mathbf{H}(f)(\mathbf{x})$
- Setze jeden kritischen Punkt $\mathbf{c}_i$ in $\mathbf{H}$ ein
- Bestimme Eigenwerte von $\mathbf{H}(f)(\mathbf{c}_i)$
- Klassifiziere nach Tabelle unten

Beispiel (2D): $f(\mathbf{x}) = x_1^2 + x_2^2$

Gradient: $\nabla f = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} = \mathbf{0} \rightarrow$ kritischer Punkt $\mathbf{c} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

Hessian: $\mathbf{H} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$, Eigenwerte: $\lambda_1 = \lambda_2 = 2 > 0 \rightarrow$ **lokales Minimum**

Zweiter Ableitungstest — Entscheidungstabelle:

Kritischer Punkt	1D: $f''(x)$	nD: Eigenwerte von $\mathbf{H}$
Lokales Minimum	> 0	Alle positiv
Lokales Maximum	< 0	Alle negativ
Sattelpunkt	$= 0$ (inconclusive)	Gemischt (mind. ein pos., ein neg.)
Inconclusive	$= 0$	Alle gleich vorzeichenig, mind. einer $= 0$

Exam Trap: Häufiger Fehler: Wenn ein Eigenwert $= 0$ ist und die anderen alle positiv (oder alle negativ), ist der Test **nicht conclusive**. Man muss höhere Ableitungen analysieren oder die Funktion direkt auswerten.

Sattelpunkte in hohen Dimensionen: Bei $n = 1'000'000$ Parametern hat $\mathbf{H}$ eine Million Eigenwerte. Schon **ein** negativer Eigenwert macht den Punkt zum Sattelpunkt — kein Minimum!

4.4 Globales Minimum finden

Tip: Vorgehen:

1. Alle kritischen Punkte bestimmen (Gradient = 0)
2. Typ bestimmen (Hessian-Eigenwerte)
3. Nur lokale Minima als Kandidaten behalten
4. Funktionswert $f(\mathbf{c}_i)$ für alle lokalen Minima berechnen
5. Kleinster Wert = globales Minimum

Sattelpunkte und Maxima können ignoriert werden — sie können nie das globale Minimum sein.

5. Newton's Method (Zweiter-Ordnung-Optimierung)

Kernidee: Statt nur den Gradienten zu nutzen, benutze auch die Hessian-Matrix um den Schritt zu skalieren — berücksichtigt die lokale Krümmung.

Update-Regel Newton's Method:

$$\mathbf{x}^{(i)} = \mathbf{x}^{(i-1)} - \mathbf{H}(f)(\mathbf{x}^{(i-1)})^{-1} \nabla_{\mathbf{x}} f(\mathbf{x}^{(i-1)})$$

Rezept: Newton's Method

1. Berechne Gradient $\nabla f(\mathbf{x})$ analytisch
2. Berechne Hessian $\mathbf{H}(f)(\mathbf{x})$ analytisch
3. Setze aktuellen Punkt $\mathbf{x}^{(i-1)}$ ein
4. Berechne $\mathbf{H}^{-1} \nabla f$ (Hessian invertieren, mit Gradient multiplizieren)
5. Update: $\mathbf{x}^{(i)} = \mathbf{x}^{(i-1)} - \mathbf{H}^{-1} \nabla f$
6. Wiederholen

Beispiel (diagonal Hessian, 2D):

$$\mathbf{H} = \begin{pmatrix} h_{11} & 0 \\ 0 & h_{22} \end{pmatrix} \rightarrow \mathbf{H}^{-1} = \begin{pmatrix} \frac{1}{h_{11}} & 0 \\ 0 & \frac{1}{h_{22}} \end{pmatrix}$$

Update für jede Komponente separat:

$$x_1^{(i)} = x_1^{(i-1)} - \frac{\frac{\partial f}{\partial x_1}}{\frac{\partial^2 f}{\partial x_1^2}}$$

Exam Trap: Newton's Method und Sattelpunkte: Newton's Method konvergiert zu kritischen Punkten — das können auch Sattelpunkte oder lokale Maxima sein! In der Übung fand Newton's Method Sattelpunkte, während Gradient Descent das globale Minimum fand. Newton's Method ist **schneller** (konvergiert in weniger Schritten), aber nicht notwendigerweise zu einem Minimum.

Vergleich Gradient Descent vs. Newton's Method:

Eigenschaft	Gradient Descent	Newton's Method
Nutzt	1. Ableitung (Gradient)	1. 1. 2. Ableitung (Hessian)
Lernrate	Manuell ε	Automatisch durch $\mathbf{H}^{-1}$
Konvergenz	Langsamer	Schneller (quadratisch)
Sattelpunktgefahr	Geringer	Höher (wird angezogen)
Rechenaufwand	Niedrig	Hoch ($\mathbf{H}$ invertieren)

6. Taylor-Approximation zweiter Ordnung

Kernidee: Jede glatte Funktion kann lokal um einen Punkt $\mathbf{x}^{(0)}$ durch ein Polynom approximiert werden.

1D:

$$f(x) \approx f(x^{(0)}) + \frac{f'(x^{(0)})}{1!}(x - x^{(0)}) + \frac{f''(x^{(0)})}{2!}(x - x^{(0)})^2$$

n-dimensional:

$$f(\mathbf{x}) \approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^T \mathbf{g} + \frac{1}{2}(\mathbf{x} - \mathbf{x}^{(0)})^T \mathbf{H}(\mathbf{x} - \mathbf{x}^{(0)})$$

mit $\mathbf{g} = \nabla_{\mathbf{x}} f(\mathbf{x}^{(0)})$ (Gradient) und $\mathbf{H} = \mathbf{H}(f)(\mathbf{x}^{(0)})$ (Hessian).

Analyse eines Gradient-Descent-Schritts

Nach einem Schritt $\mathbf{x}^{(0)} - \varepsilon \mathbf{g}$:

$$f(\mathbf{x}^{(0)} - \varepsilon \mathbf{g}) \approx f(\mathbf{x}^{(0)}) + \underbrace{-\varepsilon \mathbf{g}^T \mathbf{g}}_{\text{negativ, gut}} + \underbrace{\frac{1}{2} \varepsilon^2 \mathbf{g}^T \mathbf{H} \mathbf{g}}_{\text{Krümmungsterm}}$$

Wirkung des Krümmungsterms $\mathbf{g}^T \mathbf{H} \mathbf{g}$:

$\mathbf{g}^T \mathbf{H} \mathbf{g}$	Krümmung	Effekt auf Schritt
≤ 0	Null/negativ	Grosses ε möglich, Abstieg garantiert
> 0 , klein	Schwach positiv	Schritt verkleinert f , aber weniger als erwartet
> 0 , gross	Stark positiv	Kann f sogar erhöhen — Gradient Descent versagt!

Optimales ε (wenn $\mathbf{g}^T \mathbf{H} \mathbf{g} > 0$):

$$\varepsilon^* = \frac{\mathbf{g}^T \mathbf{g}}{\mathbf{g}^T \mathbf{H} \mathbf{g}}$$

7. Konditionszahl der Hessian und Optimierung

Kernidee: Die Konditionszahl $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ der Hessian-Matrix sagt aus, wie unterschiedlich stark die Krümmungen in verschiedene Richtungen sind.

- **Kleines κ :** Ähnliche Krümmung überall $\rightarrow$ grosses ε wählbar, schnelle Konvergenz
- **Grosses κ :** Sehr unterschiedliche Krümmungen $\rightarrow \varepsilon$ muss klein sein (wegen der stark gekrümmten Richtung) $\rightarrow$ langsamer Fortschritt in flachen Richtungen
- Das ist “poor conditioning” in der Optimierung

8. Vollständiges Lösungsrezept für Optimierungsaufgaben

Vollständiges Rezept: Von Verlustfunktion zum Minimum

Gegeben: Verlustfunktion $f(\mathbf{x})$, Startpunkt $\mathbf{x}^{(0)}$, Lernrate ε (für GD) oder Methode (Newton)

1. **Gradient berechnen:** Alle partiellen Ableitungen $\frac{\partial f}{\partial x_i}$
2. **Hessian berechnen:** Alle zweiten partiellen Ableitungen $\frac{\partial^2 f}{\partial x_i \partial x_j}$
3. **Kritische Punkte finden:** $\nabla f(\mathbf{x}) = \mathbf{0}$ lösen (oft als entkoppeltes System, wenn Kreuzableitungen = 0)
4. **Typ bestimmen:** Hessian an jedem kritischen Punkt auswerten $\rightarrow$ Eigenwerte prüfen
5. **Globales Minimum:** $f(\mathbf{c}_i)$ für alle lokalen Minima berechnen $\rightarrow$ kleinster Wert
6. **Iterative Suche (GD):** $\mathbf{x}^{(i)} = \mathbf{x}^{(i-1)} - \varepsilon \nabla f(\mathbf{x}^{(i-1)})$
7. **Iterative Suche (Newton):** $\mathbf{x}^{(i)} = \mathbf{x}^{(i-1)} - \mathbf{H}^{-1} \nabla f(\mathbf{x}^{(i-1)})$
8. **Konvergenz prüfen:** Ist gefundener Punkt ein lokales/globales Minimum? (Vergleich mit bekannten kritischen Punkten)

Exam Trap: Die häufigsten Fehler bei Optimierungsaufgaben:

1. **Gradient falsch berechnet** — besonders bei Termen mit $x_1 x_2$: Kreuzableitungen nicht vergessen!
2. **Hessian nicht symmetrisch** — bei korrekter Rechnung muss $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$ gelten
3. **Inconclusive Test ignoriert** — wenn Eigenwert = 0, ist der Test nicht abgeschlossen
4. **Newton konvergiert zu Sattelpunkt** — nicht jeder kritische Punkt ist ein Minimum!
5. **Lernrate zu gross** — der Gradient kann in die richtige Richtung zeigen, aber der Schritt überspringt das Minimum
6. **Nur lokales Minimum gefunden** — Startpunkt bestimmt, wohin GD konvergiert; für globales Minimum: mehrere Startpunkte versuchen

Week 4

Week 4: Machine Learning Basics — Generalisierung, Bias, Varianz

Lineare Regression als Einstieg

Kernidee: Wir suchen Parameter $\mathbf{w}$, die eine lineare Abbildung von Eingabe $\mathbf{x}$ auf Ausgabe y beschreiben — und das so, dass das Modell auf *neuen* Daten gut funktioniert, nicht nur auf den Trainingsdaten.

Das Modell lautet:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

Mit Bias-Trick (Spalte aus Einsen zu $\mathbf{X}$ hinzufügen) wird das zu:

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

Performance-Maß — Mean Squared Error (MSE):

$$\text{MSE}_{\text{train}} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

$$\text{MSE}_{\text{test}} = \frac{1}{m'} \sum_{i=1}^{m'} (\hat{y}^{(i)} - y^{(i)})^2$$

wobei m die Anzahl Trainingsbeispiele ist und m' die Anzahl Testbeispiele.

Tip: Woran erkenne ich: Lineare Regression?

- Aufgabe sagt “lineares Modell”, “Regression”, “Vorhersage von y aus x ”
- Gesucht: Parameter $\mathbf{w}$ (und evtl. b)
- Performance-Maß: MSE

Analytische Lösung: Nur für lineare Regression möglich — Gradient des MSE berechnen, null setzen, nach $\mathbf{w}$ auflösen:

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \text{MSE}_{\text{train}}$$

In Deep Learning: immer Gradient Descent!

Exam Trap: Häufiger Denkfehler: Bias-Term verwechseln

Der “Bias” b im linearen Modell ($\hat{y} = \mathbf{w}^T \mathbf{x} + b$) ist **nicht dasselbe** wie der statistische “Bias” eines Schätzers! Es sind zwei völlig verschiedene Konzepte mit demselben Wort.

- b im Modell = Achsenabschnitt / Intercept
- Bias des Schätzers = systematische Abweichung vom wahren Wert (siehe unten)

I.I.D.-Annahme und warum $\text{MSE}_{\text{train}}$ sinnvoll ist

Kernidee: Wenn Trainings- und Testdaten aus der *gleichen* Verteilung stammen und unabhängig sind, dann ist der erwartete Trainingsfehler gleich dem erwarteten Testfehler.

I.I.D. = Independent and Identically Distributed:

- **Unabhängig:** Jeder Datenpunkt beeinflusst keinen anderen
- **Identisch verteilt:** Alle Datenpunkte kommen aus derselben Verteilung p_{data}

Unter I.I.D.-Annahme gilt:

$$\mathbb{E}[\text{MSE}_{\text{train}}] = \mathbb{E}[\text{MSE}_{\text{test}}]$$

Deshalb: Wenn wir $\text{MSE}_{\text{train}}$ minimieren, minimieren wir im Erwartungswert auch MSE_{test} . Das ist die fundamentale Rechtfertigung für maschinelles Lernen.

Kapazität, Overfitting und Underfitting

Kernidee: Die “Kapazität” eines Modells bestimmt, wie flexible Funktionen es darstellen kann. Zu wenig Kapazität $\rightarrow$ Underfitting. Zu viel $\rightarrow$ Overfitting. Das Ziel ist der goldene Mittelweg.

Zwei Wege zu kleinem MSE_{test} :

Aus $\mathbb{E}[\text{MSE}_{\text{test}}] = \mathbb{E}[\text{MSE}_{\text{train}}] + \text{Lücke}$ folgt:

1. $\text{MSE}_{\text{train}}$ klein machen (genug Kapazität)
2. Lücke zwischen $\text{MSE}_{\text{train}}$ und MSE_{test} klein halten (nicht zu viel Kapazität)

Underfitting: Modell zu simpel

- $\text{MSE}_{\text{train}}$ groß
- MSE_{test} groß
- Lösung: Kapazität erhöhen

Overfitting: Modell zu komplex

- $\text{MSE}_{\text{train}} \approx 0$
- MSE_{test} groß
- Lösung: Kapazität reduzieren (z.B. Regularisierung)

Beispiel — Polynomielle Regression:

Modell	Parameter	Kapazität	$\hat{y} = w_1 x$	1	gering	$\hat{y} = w_1 x + w_2 x^2 + b$	3	mittel	$\hat{y} = w_1 x + \dots + w_9 x^9 + b$	10	hoch
--------	-----------	-----------	-------------------	---	--------	---------------------------------	---	--------	---	----	------

Trainings-Fehler fällt monoton mit steigender Kapazität. Test-Fehler folgt einer U-Kurve mit einem Minimum bei optimaler Kapazität.

Bayes-Fehlerrate (irreduzibler Fehler):

Selbst ein perfektes Modell mit Kenntnis von p_{data} erzielt nicht $\text{MSE}_{\text{test}} = 0$, weil Messrauschen und fehlende Kausalbeziehungen existieren. Dieser Fehler heißt **Bayes-Fehlerrate** und ist eine untere Schranke für jeden Algorithmus.

Regularisierung

Kernidee: Statt die optimale Kapazität zu suchen, baut man ein zu großes Modell und zwingt es dann durch einen Strafterm, einfachere Lösungen zu bevorzugen.

Definition:

Tip: Regularisierung ist jede Modifikation eines Lernalgorithmus, die den **Generalisierungsfehler** (aber nicht den Trainingsfehler) reduziert.

Beispiel — Weight Decay (L2-Regularisierung):

Statt nur $\text{MSE}_{\text{train}}$ zu minimieren:

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$$

- $\lambda = 0$: Kein Effekt, hohe Kapazität
- $\lambda \rightarrow \infty$: $\mathbf{w}^T \mathbf{w} \rightarrow 0$, keine Flexibilität
- Optimales λ : irgendwo dazwischen

Regularisierung — Entscheidungsregeln:

- Wenn λ groß $\rightarrow$ Modell einfacher $\rightarrow$ mehr Bias, weniger Varianz $\rightarrow$ Gefahr: Underfitting
- Wenn λ klein $\rightarrow$ Modell flexibel $\rightarrow$ weniger Bias, mehr Varianz $\rightarrow$ Gefahr: Overfitting
- λ ist ein **Hyperparameter** $\rightarrow$ wird auf dem **Validierungsset** optimiert, NICHT auf Test- oder Trainingsdaten!

Hyperparameter und Datensatz-Splits

Hyperparameter: Parameter des Lernalgorithmus, die *nicht* durch das Training gelernt werden (z.B. Polynomgrad, λ).

Warum brauchen wir drei Datensätze?

- **Trainingset:** Lernen der Modellparameter w
- **Validierungsset:** Auswahl der Hyperparameter (z.B. λ)
- **Testset:** Finale Schätzung des Generalisierungsfehlers — nur einmal benutzt!

Exam Trap: Testset darf NIEMALS zur Modellauswahl benutzt werden!

Wenn du Hyperparameter auf dem Testset optimierst, schummelt das Modell — es hat diese Daten “gesehen” und der gemessene Fehler ist kein ehrlicher Schätzer für neue Daten mehr.

Typische Aufteilung: 60% Training / 20% Validierung / 20% Test

Schätzer, Bias und Varianz

Kernidee: Maschinelles Lernen ist ein Schätzproblem — wir schätzen Parameter aus endlich vielen Daten. Bias und Varianz beschreiben, wie gut unser Schätzer dabei ist.

Notation:

- θ : wahrer (unbekannter) Parameterwert
- $\hat{\theta}_m$: Schätzer basierend auf m Datenpunkten
- $\hat{\theta}_m = g(x^{(1)}, \dots, x^{(m)})$: irgendeine Funktion der Daten

Bias eines Schätzers

$$\text{bias}(\hat{\theta}_m) = \mathbb{E}[\hat{\theta}_m] - \theta$$

- **Unverzerrt (unbiased):** $\text{bias}(\hat{\theta}_m) = 0$ für alle m
- **Asymptotisch unverzerrt:** $\lim_{m \rightarrow \infty} \text{bias}(\hat{\theta}_m) = 0$

Rezept: Bias eines Schätzers berechnen

1. Schreibe den Schätzer $\hat{\theta}_m$ explizit auf
2. Berechne $\mathbb{E}[\hat{\theta}_m]$ (Erwartungswert über alle möglichen Datensätze)
 - Linearität des Erwartungswerts ausnutzen: $\mathbb{E}[\frac{1}{m} \sum x^{(i)}] = \frac{1}{m} \sum \mathbb{E}[x^{(i)}]$
 - Wenn i.i.d.: alle $\mathbb{E}[x^{(i)}]$ sind gleich = wahrer Mittelwert
3. Berechne $\text{bias} = \mathbb{E}[\hat{\theta}_m] - \theta$
4. Falls Bias = 0: Schätzer ist unverzerrt ✓

Beispiel — Bernoulli-Parameter: Schätzer ist Stichprobenmittel $\hat{\theta}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$.

$$\mathbb{E}[\hat{\theta}_m] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}[x^{(i)}] = \frac{1}{m} \cdot m \cdot \theta = \theta$$

$$\rightarrow \text{bias} = \theta - \theta = 0 \quad \checkmark$$

Varianz und Standardfehler

$$\text{Var}(\hat{\theta}_m) = \mathbb{E}[(\hat{\theta}_m - \mathbb{E}[\hat{\theta}_m])^2]$$

$$\text{SE}(\hat{\theta}_m) = \sqrt{\text{Var}(\hat{\theta}_m)}$$

Standardfehler des Stichprobenmittels (bei Normalverteilung mit Streuung σ):

$$\text{SE}(\hat{\mu}_m) = \frac{\sigma}{\sqrt{m}}$$

Interpretation: Je mehr Datenpunkte (m groß), desto kleiner der Standardfehler $\rightarrow$ bessere Schätzung.

Beispiel: Gausssche Varianz-Schätzer

Naiver Schätzer (biased):

$$\hat{\sigma}_m^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2$$

Bias davon:

$$\text{bias}(\hat{\sigma}_m^2) = -\frac{\sigma^2}{m}$$

Unbiased Schätzer (korrigiert):

$$\tilde{\sigma}_m^2 = \frac{1}{m-1} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2$$

Tip: Woran erkenne ich, welchen Varianzschätzer ich nehmen soll?

- Teiler m : biased (systematisch zu klein), aber asymptotisch unbiased
- Teiler $m - 1$: unbiased (Bessel-Korrektur)
- In Prüfungsaufgaben: wenn “unbiased” gefordert $\rightarrow$ immer $m - 1$!

Konsistenz

Ein Schätzer ist **konsistent**, wenn:

$$\lim_{m \rightarrow \infty} \hat{\theta}_m \rightarrow \theta \quad \text{in Wahrscheinlichkeit}$$

Das bedeutet konkret:

1. Asymptotisch unverzerrt: Bias $\rightarrow 0$ für $m \rightarrow \infty$
2. Standardfehler $\rightarrow 0$ für $m \rightarrow \infty$

Bias-Varianz-Tradeoff

Kernidee: Der mittlere quadratische Fehler eines Schätzers zerfällt in zwei Teile — systematischen Fehler (Bias) und Streuung (Varianz). Man kann selten beide gleichzeitig minimieren.

MSE eines Schätzers (nicht zu verwechseln mit MSE der Regression!):

$$\mathbb{E}\left[(\hat{\theta}_m - \theta)^2\right] = \text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m)$$

Herleitung-Intuition:

$$\begin{aligned}\mathbb{E}\left[(\hat{\theta} - \theta)^2\right] &= \mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}] + \mathbb{E}[\hat{\theta}] - \theta)^2\right] \\ &= \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2\end{aligned}$$

Bias-Varianz-Tradeoff — Entscheidungsregeln:

- Niedrige Kapazität → hoher Bias, niedrige Varianz → Underfitting
- Hohe Kapazität → niedriger Bias, hohe Varianz → Overfitting
- Optimale Kapazität → bester Tradeoff → minimaler Generalisierungsfehler

Wenn Aufgabe fragt: “Modell A hat Bias 2, Varianz 1; Modell B hat Bias 0.5, Varianz 4 — welches ist besser?”

→ $\text{MSE}(\text{A}) = 2^2 + 1 = 5$, $\text{MSE}(\text{B}) = 0.5^2 + 4 = 4.25$ → Modell B ist besser

Aufgaben-Lösungsrezepte für die Prüfung

Aufgabe: Bias eines Schätzers bestimmen

Gegeben: Verteilung auf Intervall $[a, b]$, Schätzer $\hat{\mu}_m$

1. Wahren Mittelwert berechnen: $\mu = \frac{a+b}{2}$
2. Erwartungswert des Schätzers berechnen: $\mathbb{E}[\hat{\mu}_m]$
 - Linearität: Erwartungswert rein-/rausziehen
 - Integral über die Verteilung auswerten
3. Bias = $\mathbb{E}[\hat{\mu}_m] - \mu$
4. Falls Bias = 0 $\rightarrow$ unbiased $\checkmark$

Beispiel: Uniform $[-0.8, 1.2]$, Schätzer = Stichprobenmittel

$$\mu = \frac{-0.8 + 1.2}{2} = 0.2$$

$$\mathbb{E}[\hat{\mu}_m] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}[x^{(i)}] = \frac{1}{m} \cdot m \cdot 0.2 = 0.2$$

$$\text{bias} = 0.2 - 0.2 = 0 \rightarrow \text{unbiased}$$

Gleiche Verteilung, aber Schätzer nimmt Betrag:

$$\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m |x^{(i)}|$$

$$\mathbb{E}[|x|] = \int_{-0.8}^0 (-x) \frac{1}{2} dx + \int_0^{1.2} x \frac{1}{2} dx = 0.16 + 0.36 = 0.52$$

$$\text{bias} = 0.52 - 0.2 = 0.32 \rightarrow \text{biased!}$$

Aufgabe: Biased Schätzer korrigieren

Gegeben: Schätzer $\hat{\theta}$ mit bekanntem Bias $b = \mathbb{E}[\hat{\theta}] - \theta$

Methode 1 (wenn Bias bekannt):

$$\hat{\theta}_{\text{kor}} = \hat{\theta} - b$$

Methode 2 (strukturell, ohne Bias-Berechnung): Verschiebe Daten so, dass alle Werte positiv sind:

$$\hat{\mu}_{\text{kor}} = \frac{1}{m} \sum_{i=1}^m (|x^{(i)}| + C) - C$$

wobei $C > |a|$ (größer als Betrag der unteren Grenze)

Optionale Aufgabe: Max-Schätzer für Uniform $[0, \theta]$

$$\hat{\theta} = \max(x^{(1)}, \dots, x^{(m)})$$

$$\mathbb{E}[\hat{\theta}] = \frac{m}{m+1}\theta$$

$$\rightarrow \text{bias} = \frac{m}{m+1}\theta - \theta = -\frac{\theta}{m+1}$$

$$\hat{\theta}_{\text{kor}} = \frac{m+1}{m} \max(x^{(1)}, \dots, x^{(m)}) \quad \checkmark$$

No Free Lunch Theorem

Kernidee: Gemittelt über *alle* möglichen Datenverteilungen ist kein Lernalgorithmus besser als ein anderer.

Was das in der Praxis bedeutet: Wir müssen unseren Algorithmus auf die *spezifische* Art von Daten zuschneiden. Es gibt keinen universell besten Algorithmus.

Konsequenz für Deep Learning: Wir schneiden den Algorithmus auf realistische Daten zu (Bilder, Sprache, etc.) — und das funktioniert sehr gut für diese Domänen.

Tip: Übersicht: Welches Konzept bei welcher Aufgabe?

Aufgabentyp	Konzept	Schlüsselaktion	————— ————— —————	“Ist Schätzer X unbiased?”
Bias	$\mathbb{E}[\hat{\theta}] - \theta$ berechnen		“Vergleiche zwei Modelle”	Bias-Varianz
$\text{Bias}^2 + \text{Var}$ für beide		“Modell fittet Training, aber nicht Test”	Overfitting	Kapazität reduzieren / Regularisierung
		“Modell fittet weder Training noch Test”	Underfitting	Kapazität erhöhen
	“Welchen λ für Weight Decay?”	Hyperparameter	Validierungsset verwenden	
	“Warum Validierungsset?”	Testset-Integrität	Testset ist tabu für Modelauswahl	